ChatGPT

Designing a Tensor-First Planner for PDDL-Like RPG Sandbox Games

Executive Summary

The strongest design for your setting is not a pure end-to-end sequence model and not a pure classical planner. It is a **hybrid tensor-first planning system** with three layers: a **static lifted schema cache** for types, predicates, fluents, and action schemas; a **dynamic sparse grounded state** for the current world; and a **learned action-ranking / heuristic model** that sits inside symbolic search or bounded lookahead. The literature points in that direction consistently. Recent generalized-policy models such as ASNet and GABAR show that architectures aligned to action schemas or action-centric graphs can generalize from small instances to much larger ones; learned-heuristic systems such as STRIPS-HGN, GOOSE, and later lifted GNN heuristics show that learned guidance is often more scalable and reliable than direct autoregressive plan generation; and neuro-symbolic systems such as NSRTs show that symbolic outer-loop planning plus learned inner components is especially strong for long-horizon, structured tasks. [1]

For the relational-transformer part of the survey, the most relevant recent engineering reference is the public GitHub [2] repository from the Stanford Network Analysis Project [3] for Relational Transformer, together with RelBench, PluRel, and RelGT. These are **not planning models**, but they are highly relevant to your goal because they show how to encode heterogeneous schema structure directly as tensors, how to build relation-aware attention over structured fields rather than text, how to use high-throughput samplers, and how to mix local and global relational context efficiently. Their implementation stack is also useful: the Relational Transformer repo exposes a Python+Rust codebase with data preprocessing, checkpoints, and distributed training scripts; PluRel extends that stack with synthetic relational data generation; and RelGT shows a graph-transformer design with multi-element tokenization plus local/global attention that is directly inspirational for planning states and candidate actions. [4]

My main recommendation is therefore: **start with a sparse lifted/grounded hybrid representation and an action-centric GNN or graph-transformer encoder that outputs action logits and a value/heuristic estimate, then put it inside greedy best-first search or bounded A***. Use direct autoregressive plan generation only as a baseline. For a large RPG-sandbox game, correctness, applicability, and long-horizon robustness matter more than elegant sequence decoding, and current evidence suggests that search-guided learned models are more dependable than pure plan generation, especially under object renaming, atom-order symmetry, and out-of-distribution problem scaling. [5]

A final high-level conclusion is that **symbolic regression belongs in this stack as a distillation and compression tool, not the main planner**. The relevant adjacent literature uses search-enhanced transformers for symbolic regression with non-differentiable objective feedback; the planning analogue is to distill learned heuristics, action-pruning rules, or resource-feasibility formulas into compact symbolic expressions after the neural system has been trained. [6]

Literature Landscape

The planning literature most relevant to your problem clusters into five groups. First are **schema-aware generalized policy networks**, especially ASNet and its numeric extension, which exploit the relational structure of PDDL to share weights across grounded instances in the same domain. Second are **learned heuristics for symbolic search**, where STRIPS-HGN, GOOSE, later lifted GNN heuristics, and recent numeric/HTN extensions learn heuristic functions rather than whole plans. Third are **neuro-symbolic planners** such as NSRTs, Neural Logic Machines, DeepSym, and LatPlan, which combine discrete relational abstractions with learned components. Fourth are **transformer-style symbolic planners**, which include text-tokenized plans and more recent symmetry-aware training for planning transformers. Fifth is the **relational deep learning** line from the SNAP ecosystem, which is not planning-specific but offers some of the best recent ideas for tensorizing complex heterogeneous structure without text. [7]

Two broad conclusions stand out from the primary sources. One is that **inductive bias matters a great deal**. Ståhlberg, Bonet, and Geffner show that plain GNN expressivity tracks logical fragments such as C_2, which explains both where GNN policies succeed and where they fail; their later work extends the architecture toward more expressive relational reasoning, but with real scaling trade-offs. The other is that **symmetry handling is central**. Recent transformer work for planning argues that arbitrary object names and arbitrary atom order create a combinatorial explosion of equivalent inputs; their fix is symmetry-aware contrastive training, compositional tokenization, and omitting explicit positional encodings where permutation equivariance is desired. Those issues are even more important in your tensor-first setting because you are intentionally eliminating textual regularities that sometimes let sequence models "cheat." [8]

The literature also suggests that **learning a heuristic or an action ranking is often easier than learning exact value functions or full plans**. GABAR makes this explicit by arguing that ranking applicable actions in the same state is easier and often more generalizable than learning globally consistent value estimates. GOOSE similarly frames heuristic learning around graph encodings and ranking formulations rather than only pointwise regression. Recent work on LLM-generated heuristics reinforces the broader point: even when a model can synthesize useful search guidance, the strongest performance still comes from plugging that guidance into a correct symbolic search procedure instead of asking the model to plan end-to-end. [9]

For your exact objective, the best way to read the literature is therefore as a spectrum. **Most directly relevant** are ASNet, GABAR, STRIPS-HGN, GOOSE, lifted GNN heuristic work, and NSRTs. **Useful but more indirect** are DeepSym, NLM, and LatPlan, which show how to recover or manipulate symbolic abstractions from learned representations. **Good baselines but less aligned** are text-sequence planning transformers such as Plansformer and the LLM-for-PDDL line, because they rely on serialization choices you explicitly want to avoid. **A rich source of representation ideas** is the SNAP relational-model line, especially RT and RelGT. [10]

Family	Representative models	Best use in your stack	Main caution
Generalized policy networks	ASNet, numeric ASNet, GABAR	Direct action ranking or reactive policy learning within one domain family	Can fail on very long horizons without search

Family	Representative models	Best use in your stack	Main caution
Learned heuristics for search	STRIPS-HGN, GOOSE, lifted GNN heuristics, RL heuristics	Heuristic/value head for GBFS or A*	Quality depends heavily on state encoding and training states
Neuro-symbolic planners	NSRTs, DeepSym, NLM, LatPlan	Bilevel planning, abstraction learning, symbolic world-model learning	Higher engineering complexity
Transformer planners	Plansformer, symmetry-aware SymTE/SymTED	Baselines for plan generation; useful for symmetry-robust token encoders	Full quadratic attention and sequence-order sensitivity
Relational deep learning	RT, RelGT, RelBench, PluRel	Schema-aware tensorization, local/global attention, scalable preprocessing	Needs planning-specific output heads and semantics

Tensor Representation Designs

At the semantic level, classical STRIPS/PDDL tasks are built from typed objects, predicates, initial state, goals, and action schemas with preconditions and effects; numeric PDDL extends this with functions and arithmetic conditions/effects. That means your tensorization should mirror the **lifted/grounded split** of planning itself rather than flattening everything into a single token stream. The most important engineering rule is to keep **static domain tensors** separate from **dynamic state tensors** so that only a small delta changes every simulation step. [11]

Recommended canonical schema

For a first prototype, I would use the following canonical tensor dictionary. The idea is to cache everything that depends only on the domain once, and update only facts, numeric values, and candidate actions at runtime.

Static lifted tensors

- `type_id_of_object` : `[B, N]` `int16`
- `object_type_mask` : `[B, N, T]` `bool` or implicit via `type_id_of_object`
- `pred_arity` : `[P]` `uint8`
- `pred_arg_type` : `[P, Kmax]` `uint8`
- `func_arity` : `[U]` `uint8`
- `func_arg_type` : `[U, Kmax]` `uint8`
- `action_arity` : `[S]` `uint8`
- `action_arg_type` : `[S, Mmax]` `uint8`
- `pre_pred_id` : `[S, Lpre]` `int16`
- `pre_sign` : `[S, Lpre]` `int8` with `+1/-1`

- `pre_slot` : `[S, Lpre, Kmax]` `int8` mapping each predicate argument to an action parameter slot
- `eff_pred_id` : `[S, Leff]` `int16`
- `eff_op` : `[S, Leff]` `int8` with codes for add / delete / assign / increase / decrease
- `eff_slot` : `[S, Leff, Kmax]` `int8`
- `num_pre_func_id` , `num_pre_cmp` , `num_pre_rhs_kind` , `num_pre_slot` : analogous tensors for numeric preconditions
- `num_eff_func_id` , `num_eff_rhs_kind` , `num_eff_slot` : analogous tensors for numeric effects

Dynamic grounded tensors

- `true_pred_id` : `[B, F]` `int16`
- `true_pred_arg` : `[B, F, Kmax]` `int16`
- `true_pred_mask` : `[B, F]` `bool`
- `num_func_id` : `[B, Q]` `int16`
- `num_func_arg` : `[B, Q, Kmax]` `int16`
- `num_value` : `[B, Q]` `float32`
- `goal_pred_id` , `goal_pred_arg` , `goal_num_*` : same format as state, but goal-specific
- `cand_action_schema` : `[B, A]` `int16`
- `cand_action_arg` : `[B, A, Mmax]` `int16`
- `cand_action_mask` : `[B, A]` `bool`

With `N≈256–512` , `A≈100` , `Kmax≤3` , `Mmax≤4` , and `S≈16–64` , these tensors are small enough for straightforward GPU batching, especially if IDs are `int16` and embeddings use `bfloat16` or `float16` . Numeric values should stay in `float32` until stability is proven. The critical performance trick is that **facts and fluents are sparse lists**, not dense N^K cubes. That makes the runtime cost scale with the number of active facts rather than the combinatorial number of possible facts.

Four concrete encoding options

Option	Core idea	Typical tensors	Strengths	Weaknesses	Best fit
Table-style relational encoding	Represent objects, facts, fluents, schemas, and candidate actions as separate typed tables	Row-wise ID/value tensors plus table/column embeddings	Naturally matches RT-style structured attention; easy to add schema metadata	Needs careful local/global attention to avoid quadratic blow-up	Relational transformer variants

Option	Core idea	Typical tensors	Strengths	Weaknesses	Best fit
Lifted schema + sparse grounded facts	Cache lifted action/ predicate tensors; represent state as sparse true atoms and numeric assignments	Static schema tensors + sparse fact/ value lists	Most efficient and faithful to PDDL; easy incremental updates	Requires custom gather/ scatter operations	Best default for hybrid planners
Heterogeneous action-centric graph	Build object/ predicate/ action/global nodes with typed edges	`edge_index`, `edge_type`, node features, action candidates	Excellent for GNNs/action ranking; easy action masking	Harder to support very rich numeric expressions unless augmented	Best first model for policy/value
Dense low-arity predicate cubes	Use dense unary/binary tensors, sparse fallback for higher arity	`[P1,N]`, `[P2,N,N]`, plus sparse ternary+	Extremely fast for low arity; bit-packing possible	Ternary and higher explode quickly	Small or mostly-binary domains

The dense-cube option is viable only in a narrow regime. At `N=256`, a dense binary tensor with 32 predicates costs about **2.1 MB** in `uint8` or **0.26 MB** if bit-packed, which is fine. But a dense ternary tensor with only 4 predicates already costs about **67 MB** in `uint8` or **8.4 MB** bit-packed. In other words, dense encoding is a good optimization for unary/binary predicates, not a universal representation.

The representation I would actually build

I would implement a **two-level hybrid**:

1. **Lifted schema cache**
   Stores action schemas, predicate/function signatures, parameter-slot bindings, action-object type compatibility, and pre/effect templates. This is domain-constant and should live on device memory for the whole episode.

2. **Dynamic state/action layer**
   Stores currently true facts, numeric fluent values, candidate grounded actions, and goal facts. This is regenerated or incrementally updated each tick.

That hybrid gives you three large wins. It preserves PDDL structure faithfully. It makes batching easier because `S`, `Lpre`, `Leff`, `Kmax`, and `Mmax` are fixed by the domain. And it lets you update only the

sparse fact/value layer when the environment changes. That is the right trade-off for a large sandbox game where the world changes constantly but the quest/crafting/combat schema stays stable.

Model Architectures

The architectural choice should follow the representation choice. If your inputs are sparse facts and candidate actions, then the question is not “transformer or GNN?” in the abstract. It is “what computational graph gives the model the shortest path from world state to applicable action ranking or cost-to-go estimate?”

Comparative architecture view

Architecture	Best input	Natural output	Asymptotic bottleneck	What it does best	Recommendation
Action-centric GNN	Heterogeneous object/fact/action graph	Action logits, value, heuristic	Message passing over typed edges	Applicability-aware action ranking; sparse computation	Strongest first prototype
Hypergraph network	Delete-relaxation or lifted hypergraph	Heuristic/value	Hyperedge message passing	Search guidance tied to relaxed planning structure	Strong if you want learned heuristics first
Relational transformer	Table-style rows/cells, lifted+grounded typed tokens	Action logits, heuristic, plan tokens	Full or block attention	Flexible schema-aware modeling, rich global context	Good second prototype
Local-global graph transformer	Sampled subgraph tokens + centroids	Action logits, heuristic	Attention over local windows and global tokens	Better long-range context than GNN without full global attention	Best transformer family for scaling
ASNet-style schema network	Action/proposition incidence	Policy over actions	Layered schema-specific connectivity	Strong within-domain generalization with explicit planning bias	Excellent baseline
Hybrid symbolic-ML pipeline	Any of the above inside GBFS/A*/beam search	Heuristic, Q, or ranker	Search itself	Correctness, robustness, long horizon	Best overall system design

The literature gives clear support for each piece of this table. ASNet shares weights over grounded instances by following action/proposition structure; GABAR focuses on action ranking over action-centric relational graphs; STRIPS-HGN and GOOSE focus on heuristic prediction for search; RelGT shows how a graph transformer can use local plus global context efficiently; and recent symmetry-aware transformer work for planning shows that planning transformers need explicit help with object-name and atom-order symmetries. [12]

Recommended architecture

```
flowchart LR
    A[Game simulator or symbolic game wrapper] --> B[State extractor]
    B --> C[Static lifted schema cache]
    B --> D[Dynamic sparse state tensors]
    C --> E[Candidate grounding and pruning]
    D --> E
    E --> F[Action-centric graph or token builder]
    F --> G[Neural encoder]
    G --> H[Action ranking head]
    G --> I[Value or heuristic head]
    H --> J[Search or bounded lookahead]
    I --> J
    J --> K[Chosen grounded action]
    K --> A
```

This architecture does two things that matter enormously in sandbox games. It turns the model's job into **ranking a bounded candidate set** instead of discovering the entire plan distribution from scratch. And it makes search the safety mechanism that recovers from local model error. That is consistent with both the learned-heuristic literature and the most successful recent hybrid planning systems. [13]

Input and output interfaces

Your neural module should support **three interfaces**, not one:

Interface	Input	Output	Use
Policy / ranker	Current state + goal + candidate actions	Logits over `A` candidates	Reactive control, beam search, pruning
Heuristic / value	Current state + goal	Scalar `h(s)` or `V(s)`	GBFS, A*, rollout evaluation
Transition delta	State + grounded action	Changed facts/fluents or latent successor	Model-based rollouts, consistency losses

This multi-head design is superior to a single autoregressive plan decoder because it lets you reuse the same encoder for search guidance, ranking, and self-supervised transition learning. It also mirrors the

structure of modern planning systems, where one representation supports several search-time computations.

A transformer-specific warning

Full attention over all facts, objects, goals, and candidate actions becomes expensive very quickly. If you build a transformer with 8 heads in fp16, the attention-score memory alone scales quadratically:

```
xychart-beta
    title "Attention score memory per layer"
    x-axis ["600 tokens", "1200 tokens", "3000 tokens"]
    y-axis "MB" 0 --> 160
    bar [5.76, 23.04, 144.0]
```

Those numbers are schematic but directionally exact. For a planning state with a few thousand structured tokens, **full global attention is the wrong default**. If you want a transformer, follow the RelGT lesson: use local windows or sampled subgraphs, plus a small number of global centroid tokens. [14]

Training and Supervision

The most effective training strategy for this project is a **stacked supervision regime**: start from symbolic-planner traces and search states, then add self-play rollouts, then do online aggregation or RL fine-tuning. Current planning papers strongly support this progression. ASNet uses supervised guidance from teacher planners plus exploration; numeric ASNet adds a dynamic exploration strategy because numeric teacher planners are expensive; learned-heuristic papers train on states labeled by search or planner output; and RL-for-classical-planning work uses classical heuristics as dense reward shaping rather than treating the problem as sparse-reward RL from scratch. [15]

Datasets and supervision sources

You have four practical supervision streams:

Supervision source	What you collect	Why it helps
Planner traces	Optimal or satisficing action sequences, applicability masks, state deltas	Fastest way to warm-start a policy or decoder
Search frontier states	Open-list states, successor actions, heuristic labels, ranking labels	Better than trace-only data for avoiding myopic policies
Simulator rollouts	On-policy trajectories, failure states, dead ends, resource regressions	Critical for sandbox distribution shift
Synthetic generators	Procedurally generated quest/crafting/resource instances	Necessary for scale and curriculum

For public planning data, the IPC learning track is directly relevant because it formalizes the exact regime where algorithms learn offline, then solve held-out tasks in the same domain. The IPC 2023 classical dataset is also useful because it provides seven newly designed domains and generators or plan-cost information for most of them. For implementation, symbolic planners like Fast Downward and the Unified Planning library give you off-the-shelf parsing, transformation, and teacher-solver infrastructure. [16]

For game-like data, I would use three tiers. First, standard planning domains for fast iteration. Second, numeric and crafting-like domains, including MinePlanner-style large-world tasks, because those expose the object-scale and resource reasoning problems closest to sandbox games. Third, simulator-derived symbolic wrappers around benchmarks like Crafter and the NetHack [17] learning environment to stress-test exploration, long horizons, and partial observability assumptions. MinePlanner is especially relevant because it explicitly benchmarks propositional and numeric planners on large-world tasks and reports scaling failures with thousands of objects. [18]

Loss design

The model should not have a single training objective. It should optimize a weighted combination such as:

$$\mathcal{L} = \lambda_{\text{act}}\mathcal{L}_{\text{CE}} + \lambda_{\text{rank}}\mathcal{L}_{\text{listwise}} + \lambda_{\text{val}}\mathcal{L}_{\text{Huber}} + \lambda_{\text{delta}}\mathcal{L}_{\text{succ}} + \lambda_{\text{mask}}\mathcal{L}_{\text{app}} + \lambda_{\text{sym}}\mathcal{L}_{\text{sym}}$$

Where:

- `CE` is action imitation from teacher traces,
- `listwise` or pairwise ranking loss trains action ordering among applicable candidates,
- `Huber` predicts cost-to-go or search effort,
- `succ` predicts successor-state deltas,
- `app` predicts action applicability or masks impossible arguments,
- `sym` enforces object-renaming and atom-order invariance/equivariance.

That last term is not optional. The symmetry-aware transformer work makes a persuasive case that planning models should explicitly train against equivalent renamings and orderings; in a tensor-first model, I would implement this as object-ID permutation augmentation plus consistency losses on logits, hidden states, and value predictions. [19]

Curriculum and online improvement

The curriculum should progress along **object count**, **horizon**, **branching factor**, **numeric complexity**, and **quest composition depth**. ASNet-style small-to-large transfer, combined with search-state data rather than only expert trajectories, is the right starting point. After that, use DAgger-style rollout aggregation: run the current model in search, collect the states where it makes bad choices or enters dead ends, and query the teacher planner selectively on those states. Finally, use RL only after the supervised system is already competent. The RL-for-classical-planning work suggests that reward shaping from classical heuristics or residual learning over them is much more sample-efficient than raw sparse-reward learning. [20]

Scalability and Performance

The central systems problem in your setting is not only model size. It is the joint explosion of **ground facts**, **ground actions**, and **search horizon**. MinePlanner's results make this explicit: planning in large game worlds with many objects remains hard even for strong propositional and numeric planners. The learned-policy literature adds a second warning: the objectives of **optimality** and **generalization** can conflict in NP-hard domains, so chasing exact value functions is often the wrong target for large sandbox worlds. [21]

Complexity analysis

Let:

- `N` = number of entities,
- `F` = number of active facts/fluents,
- `A` = number of candidate grounded actions,
- `T` = number of tokens/nodes the model actually processes,
- `E` = number of edges in the constructed graph,
- `d` = hidden size.

Then the dominant costs are:

- **Full transformer**: compute $O(T^2 d)$, memory $O(HT^2)$
- **Block/local transformer**: compute $O(Tkd + Tgd)$ with local window `k` and global token count `g`
- **GNN**: roughly $O(Ed)$ to $O(Ed^2)$ depending on layer
- **Action readout**: $O(Ad)$
- **Higher-order relational GNNs**: can approach quadratic or cubic embedding storage and quartic message-exchange behavior when approximating richer logical fragments, which is why the more expressive relational-policy literature treats them carefully. [22]

For the game sizes you named, a practical target is:

- `N = 200–500`
- `A = 50–150` after pruning
- `F = 500–5000` active grounded facts/fluent assignments

In that regime, a **sparse action-centric GNN** is usually the safest first choice. A transformer becomes attractive only if you aggressively control `T` by using local neighborhoods, goal-conditioned token selection, or candidate-action windows.

Techniques that actually matter

The following techniques are high leverage for large RPG-sandbox worlds:

Technique	What it buys you	Why I recommend it
Static schema caching	Avoid re-encoding the domain every step	Essential in games with repeated ticks
Sparse grounded facts	Runtime proportional to active facts, not all possible facts	Prevents combinatorial blow-up
Candidate action pruning	Keep $A \approx 100$ instead of all grounded actions	Makes both GNN and search practical
Hierarchical planning	Quest-level, region-level, then tactical-level planning	Natural fit for sandbox RPGs
Delta-state updates	Update only changed fact/value rows after an action	Large speedup in simulation-heavy loops
Embedding caches	Reuse encodings for repeated or similar states in search	Important for GBFS/A*
Factored value heads	Separate resource, combat, inventory, navigation, social subvalues	Better sample efficiency and diagnostics

The **best pruning pipeline** is usually symbolic first, learned second. Symbolic type constraints, static mutual exclusions, lifted relevance, and simple resource-feasibility checks should reduce the action set before the neural ranker sees it. Then the neural model scores only the surviving candidates. This is one of the clearest ways to exploit the benefits of classical planning and machine learning together.

Hierarchy for sandbox worlds

For a large RPG-sandbox game, I would explicitly split planning into:

- **Strategic layer**: quest chains, crafting objectives, macro-resource goals,
- **Operational layer**: route, gather, equip, talk, fight, build,
- **Tactical layer**: immediate action choice in the current state.

NSRT-style bilevel planning supports the logic of this split, and open-world skill-planning work in game environments points in the same direction: long horizons become tractable when you plan over abstractions or learned skills rather than only primitive actions. [23]

Evaluation Strategy

The evaluation should be designed to answer four questions: **Does the model solve tasks? Does it generalize to larger worlds? Does it remain correct under symmetries and branching-factor shifts? And does the learned component reduce search cost or only move it around?**

Metrics

At minimum, track:

- **Solve rate / coverage**
- **Plan validity**
- **Plan cost / length**
- **Optimality gap or regret**
- **Wall-clock planning time**
- **Search expansions / generated nodes**
- **Applicable-action top-1 and top-k accuracy**
- **Heuristic calibration** against actual distance-to-go or search effort
- **OOD scaling curves** over entity count, candidate-action count, and horizon
- **Symmetry robustness** under object renaming and atom-order permutations

These are standard enough to be comparable to planning and learned-heuristic work, yet they expose the practical failure modes of game planners better than solve rate alone. [24]

Benchmarks I would use

Tier	Domains / benchmarks	Purpose
Fast synthetic classical	BlocksWorld, Logistics, Depots, Driverlog, Satellite, Zenotravel	Debug representation and search coupling
Numeric and resource-heavy	Custom crafting/resource domains, MinePlanner-style tasks	Stress resource fluents and large object sets
Recent classical competition data	IPC 2023 classical domains	Modern held-out benchmarks with varied PDDL structure
Game-like symbolic domains	Custom RPG quest/crafting/combat/ settlement generators; SimpleFPS-style NPC planning	Closest match to target product
Simulator-derived symbolic wrappers	Crafter, MinePlanner on Minecraft [25], NetHack-derived abstractions	Stress long-horizon sandbox behavior

MinePlanner is the closest public benchmark to your target, but I would not rely on it alone. It is better used as the **large-world resource/crafting tier** inside a broader benchmark suite. Crafter is useful for open-world long-horizon behavior, and older game-planning benchmarks such as SimpleFPS are useful for explicit symbolic NPC scenarios. [26]

Ablations that are worth running

The most important ablations are:

1. **Representation**

2. sparse facts vs dense low-arity cubes

3. lifted schema cache on/off
4. candidate-action nodes on/off

5. numeric fluents as nodes vs scalar side channels

6. **Architecture**

7. GNN vs local/global graph transformer

8. policy-only vs heuristic-only vs multi-head
9. action ranking vs value regression

10. symmetry loss on/off

11. **Search coupling**

12. direct policy execution

13. top-k beam without symbolic search
14. GBFS/A* with learned guidance

15. bounded lookahead with replanning

16. **Scaling**

17. train on 10–30 entities, test on 50–500

18. train with 20–40 candidate actions, test with 100+
19. train on short quests, test on multi-quest crafting chains

The key result you want is not just “higher solve rate.” It is a scaling profile showing that the learned component **reduces search effort faster than world size increases**.

Prototype Roadmap and Research Risks

A good first prototype should be intentionally narrow and engineering-heavy rather than overly ambitious. The public RT, PluRel, and Predicators codebases show that infrastructure matters: samplers, caching, preprocessing, and clean task interfaces are often the difference between an elegant idea and a usable system. The official docs for PyTorch, PyG, DGL, Unified Planning, and Fast Downward make a sensible implementation stack straightforward. [27]

Step-by-step roadmap

1. **Build the symbolic game interface**
   Define a stable internal schema for object types, predicates, numeric fluents, and action schemas.

Even if the game is not literally written in PDDL, your backend should emit STRIPS/PDDL-like structures directly as typed tensors.

2. **Implement the lifted schema cache**
   Parse domain files or internal schema definitions once. Compile action parameter-slot bindings, type constraints, and lifted pre/effect tensors.

3. **Implement sparse state tensorization**
   Convert each game tick into sparse fact lists, numeric fluent rows, and goal rows. Add incremental delta updates.

4. **Ground and prune candidate actions**
   Start with conservative symbolic pruning. Cap the neural model's action set to roughly 100 candidates.

5. **Train an action-centric GNN baseline**
   Output action logits plus a scalar heuristic/value. Use planner traces and frontier states as supervision.

6. **Integrate symbolic search**
   Use greedy best-first search first. Later add bounded A* or depth-limited replanning.

7. **Add symmetry augmentation**
   Randomly rename objects and permute atom rows during training. Enforce consistency on logits and heuristic predictions.

8. **Add a local/global transformer variant**
   Reuse the same tensorizer, but tokenize only selected fact/action neighborhoods and a few global tokens.

9. **Scale to numeric crafting and sandbox quests**
   Add resource fluents, recipe graphs, loadout reasoning, and region-level hierarchy.

10. **Distill and compress**
    Use symbolic regression or explicit rule extraction to distill pruning rules, resource-feasibility checks, or heuristic residuals into compact interpretable formulas.

Encoding pseudocode

```
from dataclasses import dataclass
import torch

@dataclass
class PlanningBatch:
    # static / cached per domain
    pred_arity: torch.Tensor          # [P]
```

```
    pred_arg_type: torch.Tensor       # [P, Kmax]
    act_arity: torch.Tensor           # [S]
    act_arg_type: torch.Tensor        # [S, Mmax]
    pre_pred_id: torch.Tensor         # [S, Lpre]
    pre_sign: torch.Tensor            # [S, Lpre]  (+1 / -1)
    pre_slot: torch.Tensor            # [S, Lpre, Kmax]
    eff_pred_id: torch.Tensor         # [S, Leff]
    eff_op: torch.Tensor              # [S, Leff]
    eff_slot: torch.Tensor            # [S, Leff, Kmax]

    # dynamic / per problem instance
    obj_type: torch.Tensor            # [B, N]
    true_pred_id: torch.Tensor        # [B, F]
    true_pred_arg: torch.Tensor       # [B, F, Kmax]
    true_pred_mask: torch.Tensor      # [B, F]
    num_func_id: torch.Tensor         # [B, Q]
    num_func_arg: torch.Tensor        # [B, Q, Kmax]
    num_value: torch.Tensor           # [B, Q]
    goal_pred_id: torch.Tensor        # [B, G]
    goal_pred_arg: torch.Tensor       # [B, G, Kmax]
    goal_pred_mask: torch.Tensor      # [B, G]
    cand_action_schema: torch.Tensor  # [B, A]
    cand_action_arg: torch.Tensor     # [B, A, Mmax]
    cand_action_mask: torch.Tensor    # [B, A]

def encode_problem(problem, domain_cache, Nmax=512, Fmax=4096, Amax=128):
    """
    Convert one symbolic planning instance into padded sparse tensors.
    `problem` can come from internal game state or a PDDL parser.
    """
    B = 1
    Kmax = domain_cache.Kmax
    Mmax = domain_cache.Mmax

    obj_type = torch.full((B, Nmax), -1, dtype=torch.int16)
    for i, obj in enumerate(problem.objects[:Nmax]):
        obj_type[0, i] = domain_cache.type_to_id[obj.type_name]

    true_pred_id = torch.full((B, Fmax), -1, dtype=torch.int16)
    true_pred_arg = torch.full((B, Fmax, Kmax), -1, dtype=torch.int16)
    true_pred_mask = torch.zeros((B, Fmax), dtype=torch.bool)

    for j, atom in enumerate(problem.true_atoms[:Fmax]):
        true_pred_id[0, j] = domain_cache.pred_to_id[atom.predicate]
        for k, arg in enumerate(atom.args):
            true_pred_arg[0, j, k] = problem.object_to_id[arg]
        true_pred_mask[0, j] = True
```

```
    cand_action_schema = torch.full((B, Amax), -1, dtype=torch.int16)
    cand_action_arg = torch.full((B, Amax, Mmax), -1, dtype=torch.int16)
    cand_action_mask = torch.zeros((B, Amax), dtype=torch.bool)

    # candidate action generation should already include symbolic pruning
    for a_idx, act in enumerate(problem.candidate_actions[:Amax]):
        cand_action_schema[0, a_idx] = domain_cache.act_to_id[act.schema]
        for m, arg in enumerate(act.args):
            cand_action_arg[0, a_idx, m] = problem.object_to_id[arg]
        cand_action_mask[0, a_idx] = True

    return {
        "obj_type": obj_type,
        "true_pred_id": true_pred_id,
        "true_pred_arg": true_pred_arg,
        "true_pred_mask": true_pred_mask,
        "cand_action_schema": cand_action_schema,
        "cand_action_arg": cand_action_arg,
        "cand_action_mask": cand_action_mask,
    }
```

Forward-pass pseudocode

```
import torch
import torch.nn as nn

class ActionCentricPlanner(nn.Module):
    def __init__(self, num_types, num_preds, num_actions, d_model=256):
        super().__init__()
        self.type_emb = nn.Embedding(num_types + 1, d_model)
        self.pred_emb = nn.Embedding(num_preds + 1, d_model)
        self.act_emb = nn.Embedding(num_actions + 1, d_model)

        self.obj_mlp = nn.Sequential(
            nn.Linear(d_model, d_model),
            nn.ReLU(),
            nn.Linear(d_model, d_model),
        )

        self.fact_mlp = nn.Sequential(
            nn.Linear(3 * d_model, d_model),
            nn.ReLU(),
            nn.Linear(d_model, d_model),
        )

        self.action_mlp = nn.Sequential(
```

```
            nn.Linear(2 * d_model, d_model),
            nn.ReLU(),
            nn.Linear(d_model, d_model),
        )

        self.encoder = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(
                d_model=d_model,
                nhead=8,
                dim_feedforward=4 * d_model,
                batch_first=True,
            ),
            num_layers=4,
        )

        self.action_head = nn.Linear(d_model, 1)
        self.value_head = nn.Linear(d_model, 1)

    def forward(self, batch):
        """
        Sketch:
        1. embed object ids/types
        2. build fact embeddings by gathering argument object embeddings
        3. build candidate action embeddings from schema + argument object pool
        4. run shared encoder on concatenated sparse tokens
        5. read out action logits and state value
        """
        obj_x = self.type_emb(batch["obj_type"].clamp_min(0).long())
        obj_x = self.obj_mlp(obj_x)

        pred_x = self.pred_emb(batch["true_pred_id"].clamp_min(0).long())
        arg0 = obj_x.gather(
            1,
            batch["true_pred_arg"][...,
0].clamp_min(0).unsqueeze(-1).expand(-1, -1, obj_x.size(-1))
        )
        arg1 = obj_x.gather(
            1,
            batch["true_pred_arg"][...,
1].clamp_min(0).unsqueeze(-1).expand(-1, -1, obj_x.size(-1))
        )
        fact_x = self.fact_mlp(torch.cat([pred_x, arg0, arg1], dim=-1))

        act_schema_x =
self.act_emb(batch["cand_action_schema"].clamp_min(0).long())
        act_arg0 = obj_x.gather(
            1,
            batch["cand_action_arg"][...,
```

```
0].clamp_min(0).unsqueeze(-1).expand(-1, -1, obj_x.size(-1))
        )
        action_x = self.action_mlp(torch.cat([act_schema_x, act_arg0], dim=-1))

        tokens = torch.cat([obj_x, fact_x, action_x], dim=1)
        enc = self.encoder(tokens)

        num_actions = action_x.size(1)
        action_tokens = enc[:, -num_actions:, :]
        action_logits = self.action_head(action_tokens).squeeze(-1)

        state_token = enc.mean(dim=1)
        value = self.value_head(state_token).squeeze(-1)
        return action_logits, value
```

Risks and open research questions

The largest risk is that **grounding, not neural inference, becomes the real bottleneck**. If the game world yields tens of thousands of candidate actions per tick, you will need stronger lifted pruning or hierarchical decomposition before any model architecture matters. A second risk is that the learned model overfits to procedural generators and fails on authored quests or edge-case resource interactions. A third is that purely classical assumptions break: many sandbox games are partially observable, concurrent, stochastic, or event-driven in ways closer to temporal or numeric planning than STRIPS. [28]

The main open research questions are these. How much lifted reasoning should happen **before** grounding candidate actions? Can an RT/RelGT-style local-global transformer outperform action-centric GNNs once attention is restricted to small structural neighborhoods? Can learned abstractions in the spirit of NSRTs be adapted from robotics to large game worlds with inventories, crafting, NPC relations, and region-level traversal? And can symbolic regression reliably distill useful neural heuristics into compact constraints or formulas without collapsing performance? The literature gives promising hints, but it does not yet settle these questions for large RPG-sandbox planning. [29]

---

[1] [3] [7] [10] [12] [15] [20] Action Schema Networks: Generalised Policies with Deep Learning
https://arxiv.org/abs/1709.04271?utm_source=chatgpt.com

[2] [14] Relational Graph Transformer
https://arxiv.org/html/2505.10960v2

[4] [27] https://github.com/snap-stanford/relational-transformer
https://github.com/snap-stanford/relational-transformer

[5] [19] https://mrlab.ai/papers/fritzsche-et-al-aaai2026a.pdf
https://mrlab.ai/papers/fritzsche-et-al-aaai2026a.pdf

[6] https://arxiv.org/abs/2303.06833
https://arxiv.org/abs/2303.06833

8 https://arxiv.org/abs/2109.10129
https://arxiv.org/abs/2109.10129

9 https://arxiv.org/html/2412.04752v2
https://arxiv.org/html/2412.04752v2

11 https://ai.stanford.edu/~nilsson/OnlinePubs-Nils/PublishedPapers/strips.pdf
https://ai.stanford.edu/~nilsson/OnlinePubs-Nils/PublishedPapers/strips.pdf

13 17 https://arxiv.org/abs/2312.11143
https://arxiv.org/abs/2312.11143

16 https://ipc2023-learning.github.io/
https://ipc2023-learning.github.io/

18 21 26 https://arxiv.org/abs/2312.12891
https://arxiv.org/abs/2312.12891

22 https://arxiv.org/abs/2403.11734
https://arxiv.org/abs/2403.11734

23 29 https://arxiv.org/abs/2105.14074
https://arxiv.org/abs/2105.14074

24 25 https://icaps23.icaps-conference.org/competitions/
https://icaps23.icaps-conference.org/competitions/

28 https://arxiv.org/abs/1106.4561
https://arxiv.org/abs/1106.4561